RevenueLens

AI revenue-assurance plug-in for legacy telecom billing

Synthetic-data demo

Portfolio project by Ajay Kumar Kondeti

The problem

Revenue assurance still means checking bills by hand

Revenue quietly leaks

Money the bills should have collected never arrives, and nobody notices it is missing.

Unbilled usage

Wrong tariffs

Expired discounts

Slow

Checks run long after the bill goes out, so a leak is found weeks later.

Misses things

A sample cannot cover every bill, and checking by hand adds mistakes.

A plug-in on top, not a replacement

Plug in, never replace

Sits on top of the billing system you already run. Nothing gets replaced.

Proof with every finding

Cites the exact records behind it. The money math runs in code.



Read-only first

Day one it only reads. It changes nothing in billing.

Humans approve every fix

Fixes stay drafts until a person approves. Trust comes before any write access.

How it learns

Learns your billing, then proves it

Every rule it learns cites its source: a table or config row, billed-data evidence, a ticket, or a page.

Proof: bill replay

Recalculates past bills from the learned rules and compares them with the real bills.

When sources disagree, the backend and real bills win, and a human decides.

Billed data: what was actually charged

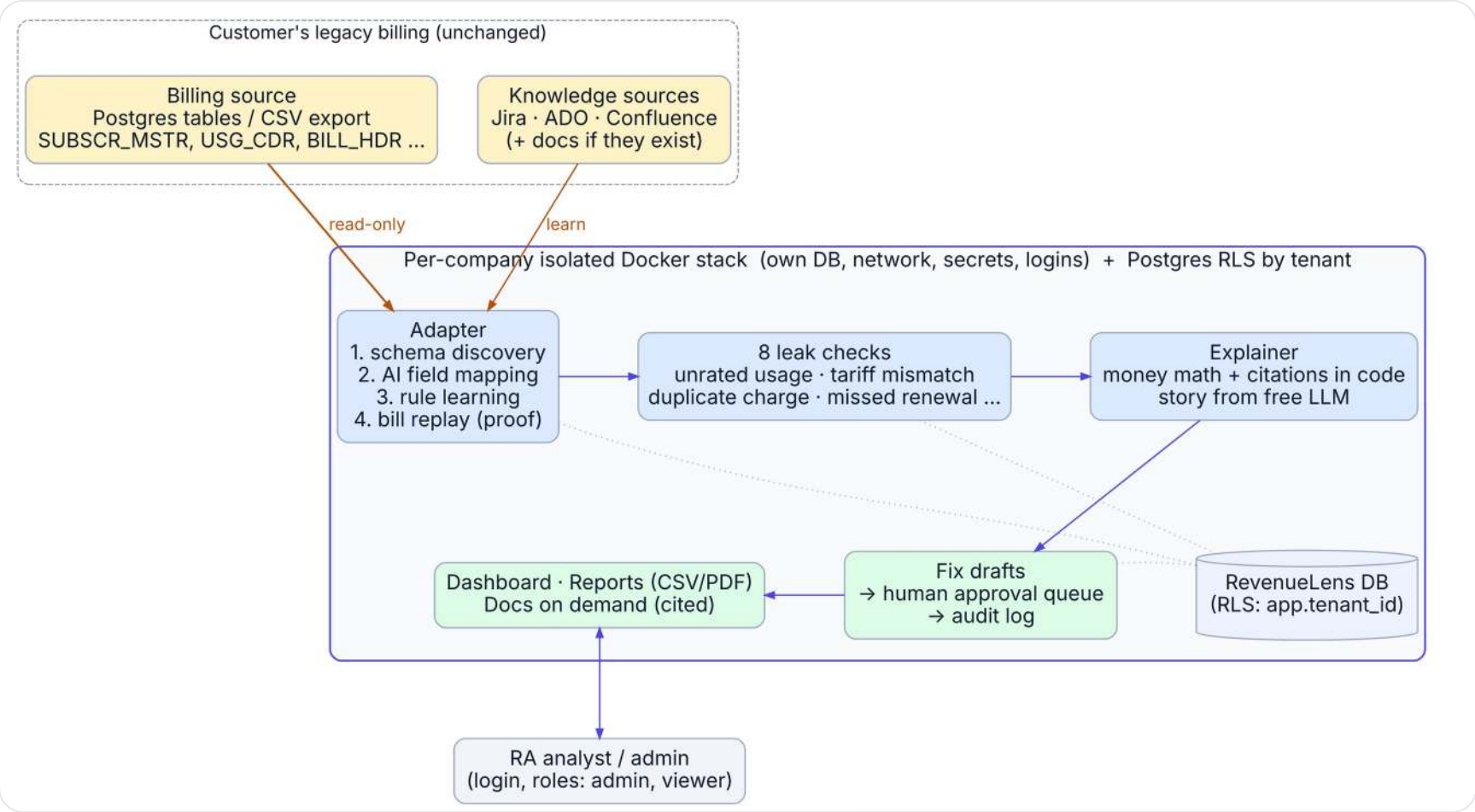
Backend tables and configs

Schema discovery reads tables, columns and keys; AI maps each field to usage, invoices, payments or catalog.

Jira and ADO tickets, Confluence pages

Existing docs, when available

Read-only in, human-approved fixes out



Legacy billing unchanged. Billing tables and knowledge sources are read, never replaced.

One pipeline. Adapter, 8 leak checks, explainer, then fix drafts in a human approval queue with an audit log.

Isolated per company. Own Docker stack plus Postgres row-level security by tenant.

Leak detection

Eight leak checks, each with proof

Money math and citations are computed in code; the story is written by a free LLM.

01

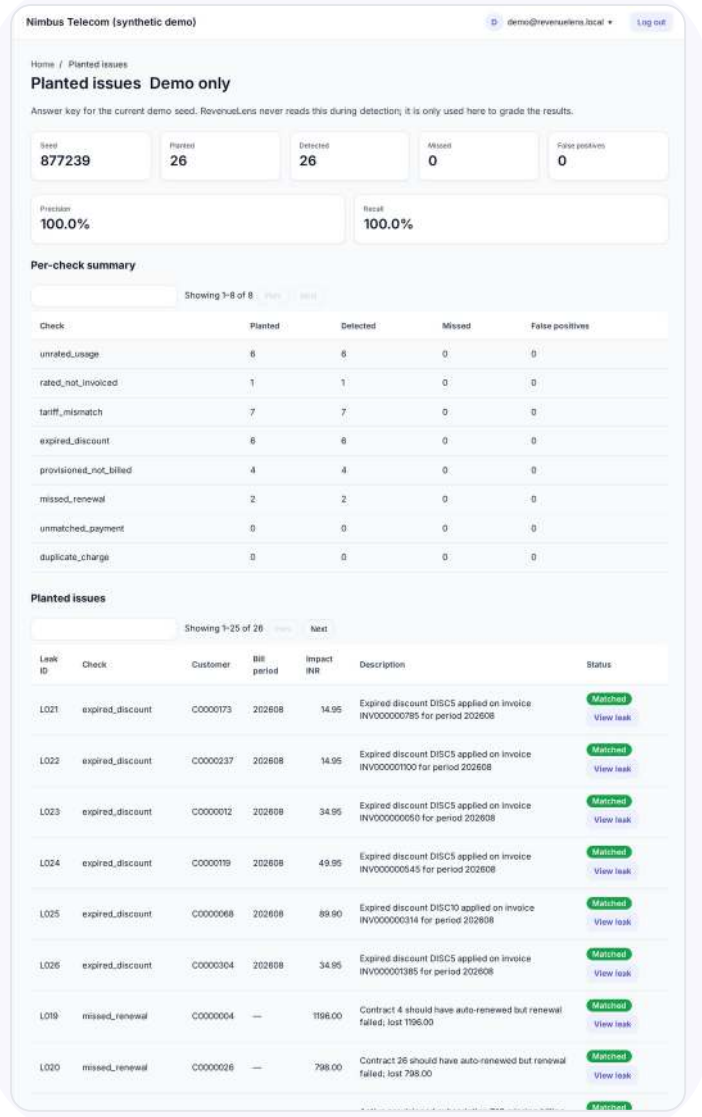
Usage and billing

- Unrated usage
- Rated, not invoiced
- Tariff mismatch
- Expired discount

02

Accounts and payments

- Provisioned, not billed
- Missed renewal
- Unmatched payment
- Duplicate charge



— Fixes

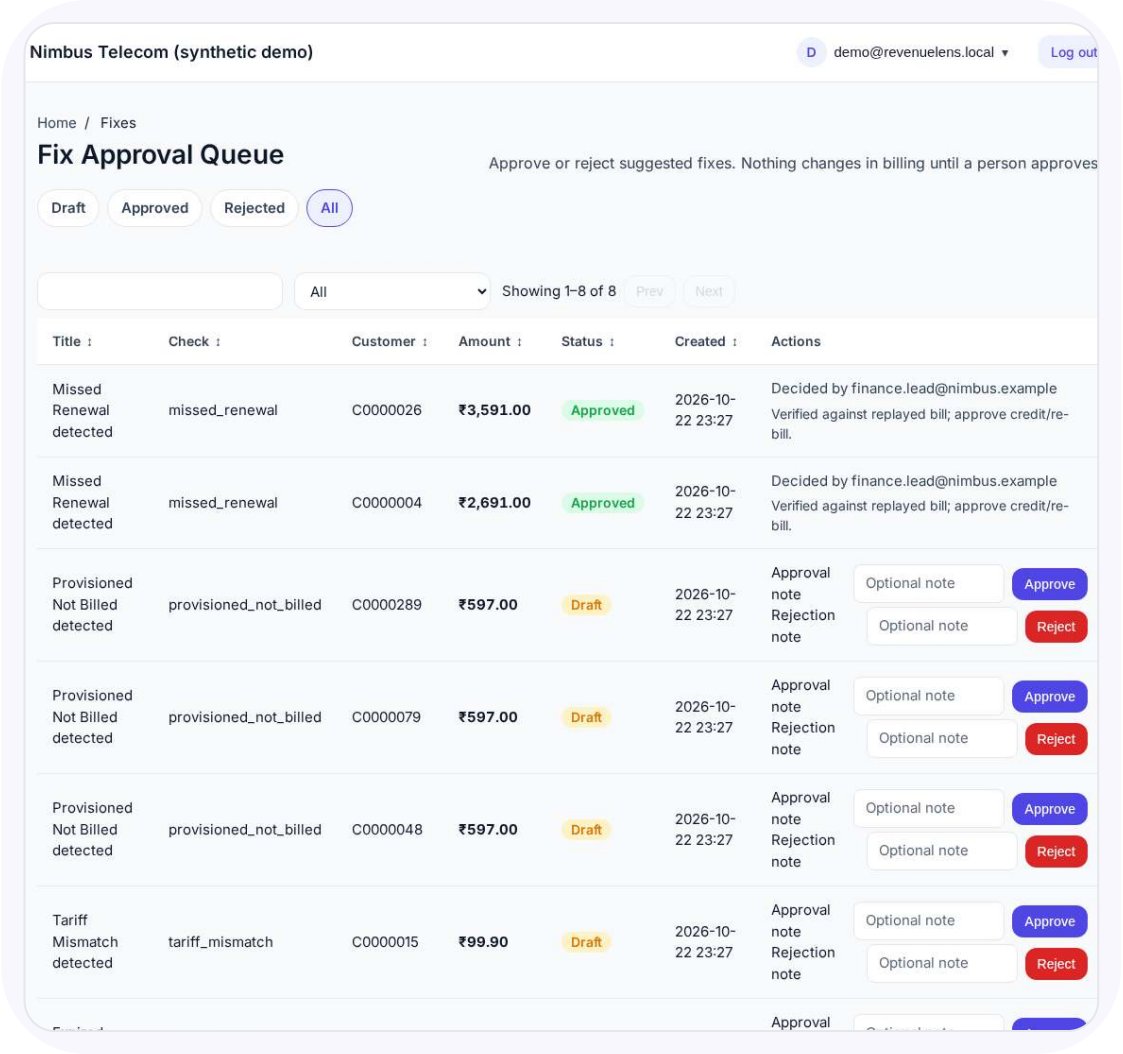
Every fix waits for a human

Drafted, then queued

A fix is drafted for each leak and goes to an approval queue. Admins and analysts approve or reject it; viewers cannot.

Every action in the audit log

Who did what, and when. Billing changes only after a person approves.



— Isolation and security

One company, one isolated stack

Each company gets its own Docker stack: its own database, network, secrets and logins. One stack cannot reach another.

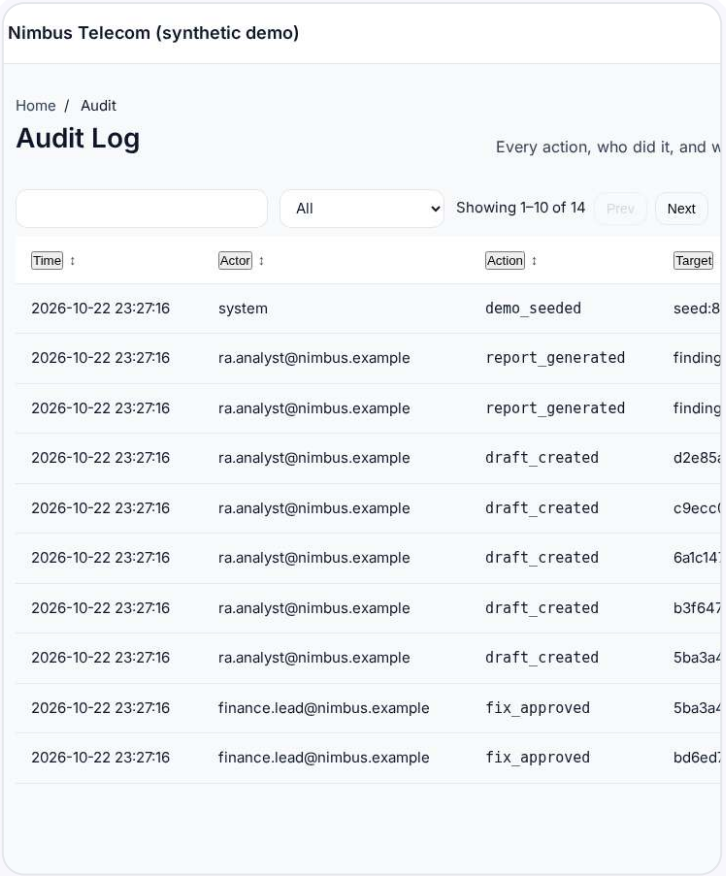
Postgres row-level security sits underneath as a second layer.

Per-instance secrets

Own JWT secret per stack; passwords hashed with bcrypt.

Proven in CI

A CI end-to-end test proves cross-company tokens are rejected.



Audit log: every action, who did it, and when.

Results on synthetic data

100%

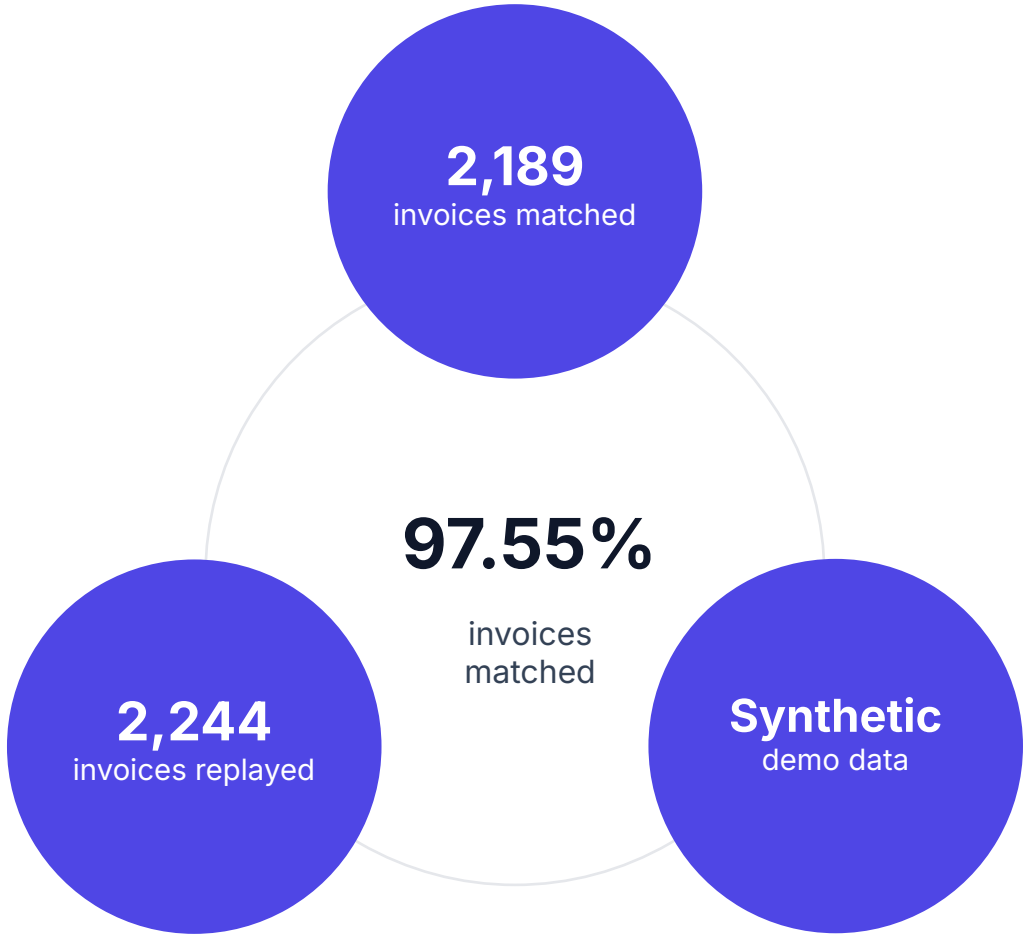
Recall

100%

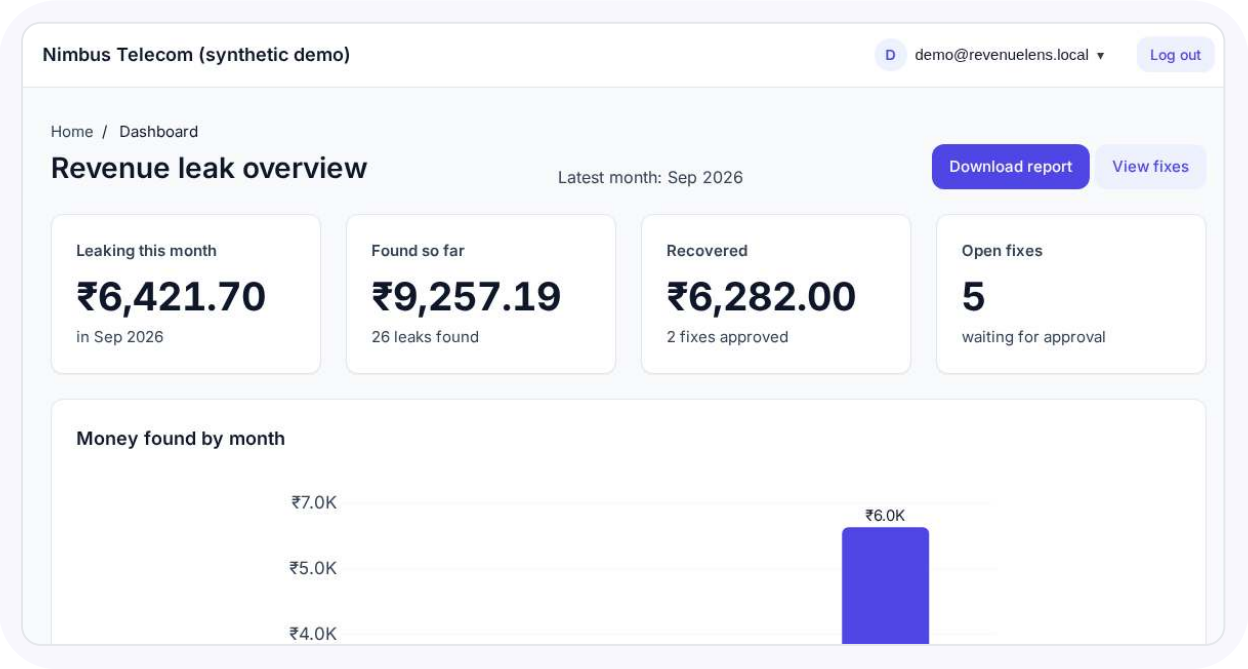
Precision

Every randomly planted leak found, with no false alarms, across many seeds.

All numbers come from the demo's generated billing data, not a real telecom.



Try it in one command



Run the demo

- **docker compose -f docker-compose.demo.yml up**
- Then open localhost:8765
- github.com/ajay206/revenuelens